Optimizing Small Language Models (SLMs) for Syntax-Agnostic, Resource-Efficient C/C++ Vulnerability Detection

Jonathan Shanmuganantham
University of Waterloo

Abstract

Small language models (1B to 12B parameters) can run on a developer's own machine or CI runner, which makes them a cheap and private alternative to sending code to a hosted frontier model. This paper asks how close they can get. We fine-tune five open models for C/C++ vulnerability detection and compare them with a frozen frontier model, gpt-5.6-luna, on both source code and compiler-generated LLVM IR. We test on two benchmarks we built. Synthset has 957 synthetic programs whose labels were confirmed by actually running them, and Realset has 2,995 real CVE functions paired with their fixes, none of which appear in training. Hosted APIs expose no output probabilities, so the frontier model cannot be distilled in the classic sense. Training instead uses rationale SFT, supervised fine-tuning on written rationales from Claude Opus that are kept only when their verdict matches the verified label, followed by a preference-optimization stage, all on single NVIDIA L40S (48 GB) GPUs on AWS for $404 in total. On clean synthetic code, a fine-tuned 9B model matches the frontier (balanced accuracy 0.626 against 0.629) with one third of its false positives and better weakness naming (0.821 against 0.765). Real code is much harder. Every small model scores at chance (0.50 to 0.51), and the frontier only reaches 0.545. Even so, the small models are better at telling a vulnerable function from its patched version (pair-consistency 0.226 against 0.156) and at naming the weakness family (0.44 against 0.36), and a 2.6B model names real weaknesses about as well as the frontier. Compiling to LLVM IR does not help the small models, which detect and name better on source. IR helps only the frontier, where it cuts false positives at the cost of missing more real bugs. The largest real-code gain comes from rationale SFT, which raises pair-consistency from 0.099 to 0.209 (training on verdicts alone reaches 0.141). Preference optimization makes the model more cautious without making it more accurate. No model is reliable enough to run alone on real code, so we recommend a two-step cascade, where one model flags code and another names the weakness, with a human reviewer checking the results. The generator, oracles, training and evaluation code, datasets and checkpoint manifest are available at https://github.com/jshan9078/cwe-model-training.

1 Introduction and Motivation

Manual code review cannot keep up with how fast large teams ship code. Static analysis tools such as SonarQube, CodeQL and Semgrep find flaws deterministically, but their false-positive rates can exceed 68%, which buries reviewers in alerts. Hosted large language models catch more real bugs, but every review sends private code to a third party and costs an API call.

Small language models (1B to 12B parameters) can instead run locally inside a CI pipeline, which keeps code private and the cost fixed. Their weakness is fragility. A model trained on raw C/C++ will often change its verdict after a cosmetic edit, such as renaming a variable or rewriting a loop. Interpretability work finds that vulnerability reasoning lives in a small set of model components [7], which suggests that targeted fine-tuning on cleaner inputs and on written-out reasoning could make small models less sensitive to surface details.

We test this idea in three stages.

1. **Change the input representation.** Compile C/C++ to LLVM IR, which strips formatting and comments and renames every variable into numbered registers, and check whether that removes the surface noise small models trip on.
2. **Fine-tune on teacher rationales (rationale SFT).** Train small models on explanations written by frontier models, keeping only explanations whose verdict matches the verified label, together with real CVE examples.
3. **Preference optimization.** Run DPO on pairs of correct and incorrect verdicts, with correctness decided by executing the program under sanitizers.

1.1 Research Questions

The study answers three research questions.

- **RQ1** How far do 1 to 12B on-device models trail gpt-5.6-luna (high) on CWE classification, measured by balanced accuracy, naming, and pair-consistency, and how does the gap scale with model size and architecture?
- **RQ2** Does compiler-derived LLVM IR help or hurt a small model, relative to source, for the same task? Running gpt-5.6-luna (high) on IR as a control separates a benefit specific to small models from IR simply being easier for every model.
- **RQ3** How much does supervised fine-tuning on teacher rationales (rationale SFT), and preference-based reinforcement learning on top of it, close the gap over the untuned baselines?

1.2 Related Work

Thirteen recent papers frame this work. We group them into five themes.

1.2.1 SLM and LLM Vulnerability Benchmarks

Müller-Uri and Pykälistö [8] compared local models (CodeLlama-7B, Qwen3-1.7B, Mistral-7B, DeepSeek-Coder-6.7B, Llama-3.2-3B) with cloud models (Claude Opus 4.7 and

ChatGPT-5.4) on PrimeVul. They found that fine-tuning mostly teaches a model the expected output format and adds little real security reasoning, and that few-shot examples do almost nothing for models that are already fine-tuned.

Lin and Mohaisen [9] evaluated 14 models on Juliet and DiverseVul. Larger models were not reliably better, and small models had particular trouble choosing among many weakness classes. The effect of quantization varied from model to model.

Henareh [10] fine-tuned Llama-3.1-8B on general cybersecurity instruction data. Without code-specific examples, accuracy improved by only 1.4% to 1.6% and reasoning got worse, which suggests that tuning for vulnerability detection needs real code in the training data.

1.2.2 Circuit Analysis and Mechanistic Interpretability

Al Atiiq et al. [7] traced the internal circuits Gemma-2-2B uses when it detects vulnerabilities. Only 2 of its 26 layers did the classification work, and switching off the most selective neurons cut detection performance by more than half.

1.2.3 Reinforcement Learning for Vulnerability Detection

Lu et al. [11] introduced DAGVUL, which trains Qwen3-8B with reinforcement learning from verifiable rewards (GRPO) and rewards the model for reasoning about logical dependencies. It reached 70.97% zero-shot accuracy, a 17% gain over the base model, and 76.11% with chain-of-thought prompting.

Rondanini et al. [12] used LoRA adapters for continual learning in malware detection on edge devices. Adapters that added under 1% more parameters improved accuracy by 20% to 25%.

Kozachok and Kozachok [13] surveyed small language models for generating code and domain-specific languages, and organized parameter-efficient fine-tuning methods by the kind of structured representation they target.

1.2.4 Code Representations and SAST Integration

Gnieciak and Szandala [5] compared GPT-4.1, Mistral Large and DeepSeek V3 with SonarQube, CodeQL and SnykCode. The language models had higher F1 scores, but they also produced more false positives and made errors caused by how code was tokenized.

Hu et al. [14] built QLPro, which pairs an LLM with CodeQL in a loop of three cooperating roles. The rules it wrote compiled 90.9% of the time, and it found 66.1% of vulnerabilities.

Jin [15] surveyed five ways of adapting LLMs to vulnerability detection and found that decoder-only models tend to be precise while missing more bugs.

Xiong and Zhang [16] used LLM agents to filter static-analysis alerts and cut the false-positive rate from above 98% to 6.3%.

1.2.5 Statement-Level and Paired Benchmarks

Ahmed et al. [17] introduced SECVULEVAL, a benchmark that asks models to point to the exact vulnerable statements across 25,440 functions from 5,867 CVEs. Open-source models reached only 7.14% to 13.57% statement-level F1.

Li et al. [6] re-evaluated pre-trained code models on PrimeVul and found that their performance drops sharply when control flow is restructured or comments are added.

2 Data

The data comes in two parts. The synthetic part has 14,166 C/C++ samples across 24 weakness classes, built so that surface features such as variable names carry no information about the label. Every synthetic label is confirmed by compiling and running the program under a matching runtime check (AddressSanitizer, UndefinedBehaviorSanitizer, MemorySanitizer, ThreadSanitizer, or an exploit harness), so a sample is only called vulnerable if the bug actually fires. The real-world part has 2,995 minimal-diff CVE pairs from BigVul and 2,481 pairs from PrimeVul, where each pair is a vulnerable function and its fixed version. For fine-tuning, we also collected 638 written rationales for source code (637 for LLVM IR) from frontier models, keeping only rationales whose verdict matched the verified label.

2.1 Real-World Datasets

Three public datasets supply real, human-written code.

- **BigVul (Fan et al.) [1]** Functions from real C/C++ GitHub projects, captured before and after CVE-fixing commits. From it we build **Realset**, a clean evaluation set of 2,995 vulnerable/fixed pairs (5,990 functions) that provably shares nothing with the training data. The median fix changes only 9% of a function's lines.
- **PrimeVul (Ding et al.) [2]** A cleaned CVE dataset. We use 3,128 of its examples, deduplicated by hash against every evaluation set, to fine-tune the real-code model **REALSFT**.
- **SARD Juliet C/C++ Suite v1.3 [3]** A standard benchmark suite, used only to add 7,980 training samples and never for evaluation.

2.2 Synthetic Data

The generator builds each program by placing one of 104 defect patterns inside one of 50 hand-written host programs, then varies the result along 16 independent surface dimensions such as identifier names, buffer sizes, allocators and control-flow structure. Each sample is created as a twin pair, one vulnerable and one secure. A coin flip decides which twin is released, so a model that only counts tokens cannot beat chance.

2.2.1 Oracle Verification

No synthetic label is accepted until a runtime oracle for its weakness class confirms it. Memory errors such as overflows and use-after-free must produce the matching AddressSanitizer report. Integer errors must trigger UndefinedBehaviorSanitizer, uninitialized reads must trigger MemorySanitizer, and concurrency bugs must trigger ThreadSanitizer. Injection flaws must be confirmed by an exploit harness.

Table 1 shows the Python code that turns AddressSanitizer and UndefinedBehaviorSanitizer reports into labels.

2.3 Evaluation Benchmarks

All models are evaluated on two benchmarks that share no programs.

- **Synthset (synthetic benchmark)** 957 oracle-verified C/C++ programs across 24 weakness classes, balanced at 20 per class. Statistical probes on surface features score at chance, so a high score has to come from reasoning about what the

Table 1: Dynamic Oracle Verification Parser Code

Oracle Sanitizer Parser Implementation (Python)

```
def parse_asan_report(output: str) -> dict:
    if "heap-buffer-overflow" in output: kind = "CWE-122"
    elif "stack-buffer-overflow" in output: kind = "CWE-121"
    elif "heap-use-after-free" in output: kind = "CWE-416"
    elif "attempting double-free" in output: kind = "CWE-415"
    else: kind = "UNKNOWN"
    is_w = "WRITE on unknown address" in output
    is_r = "READ on unknown address" in output
    return {"cwe": kind, "write": is_w, "read": is_r,
            "valid": kind != "UNKNOWN"}

def verify_ubsan_integer(output: str) -> str:
    if "unsigned integer overflow" in output:
        return "CWE-190"
    if "shift out of bounds" in output:
        return "CWE-369"
    return "SECURE"
```

code does.

- **Realset (real-world benchmark)** 2,995 real CVE pairs (5,990 functions) from BigVul, provably disjoint from all training data. It tests whether a model can tell a real vulnerable function from the same function after its one-commit fix.

For the direct source-versus-IR comparison we also define **Deployable-733**, the 733 Synthset programs whose LLVM IR fits in the 16,384-token training window. The other 224 programs are too long in IR form and are dropped from IR training (Section 4.3). Scoring both representations on the same 733 programs keeps that filtering from favoring either side, which makes the head-to-head in Section 4.4 fair.

2.4 Rationale SFT

The training rationales come from frontier teacher models (Claude Opus 4.8 and Opus 5 at high reasoning effort). We call the procedure rationale SFT and avoid calling it distillation. Classic knowledge distillation [18] trains the student p_θ to match the teacher's full next-token distribution p_T at every position, which requires the teacher's output probabilities. A hosted API exposes no logits, internal activations or hidden reasoning, so that is impossible here. Instead, for each program x with oracle label $g(x)$ we sample a teacher output $y = (r, v) \sim p_T(\cdot \mid x)$, made of a written rationale r and a verdict v, keep it only when $v = g(x)$, and train the student by maximum likelihood on the kept outputs.

$$\mathcal{L}_{\text{RSFT}}(\theta) = -\,\mathbb{E}_x\,\mathbb{E}_{y\sim p_T(\cdot|x,\,A)}\big[\log p_\theta(y \mid x)\big], \quad A = \{v(y) = g(x)\} \tag{1}$$

Since cross-entropy differs from KL divergence only by the entropy of the target, minimizing this loss minimizes the sequence-level forward KL from the correctness-filtered teacher $p_T(\cdot \mid x, A)$ to the student, estimated from a single accepted sample per program. This matches what Kim and Rush [19] call sequence-level knowledge distillation, with a verifier filter added. It departs from classic distillation in three ways. The student sees one sampled output where classic distillation uses a full distribution, it imitates only the teacher's correct outputs, and the rationale is the explanation the teacher writes out, which need not reflect the hidden reasoning that produced its verdict. Each teacher call runs in its own isolated subagent in the Claude Code harness and sees one program, with no label and no other samples.

Figure 1 shows the pipeline. A rationale enters the training set only when its verdict matches the oracle label, and is discarded otherwise.

Table 2: Exact Teacher Prompt and Sample JSON Output

Teacher System Prompt

```
You are a senior security engineer reviewing production C/C++ (or LLVM IR)
before it ships.  Follow the values that bound each access, allocation,
release or interpreter call.  State which specific operation is or is not safe
and why, quoting concrete symbols.  SIBLING DISCRIMINATION (mandatory) when
you choose a CWE you MUST name the single closest sibling from the list and
cite the one concrete fact that rules it out.  Choose the verdict ONLY from
the closed CWE list, or SECURE. Answer ONLY as JSON with two keys {"think":
"<120-300 words reasoning>", "verdict":  "<CWE-XXX or SECURE>"}.
```

Sample Teacher Output

```
{"think":  "The function allocates buffer iTemRef using new
short[FIELDREF+CHUNKREF]. At line 14, iTemRef is set to NULL before calling
delete[] iTemRef.  Calling delete[] on a NULL pointer is a no-op, leaving
the previously allocated memory orphaned on the heap without deallocation.
This constitutes an unreleased resource flaw.  Sibling 415 (double free) is
ruled out because free/delete is never called twice on the valid pointer.
Sibling 416 (use-after-free) is ruled out because the memory is never accessed
post-nullification.  Therefore this is a heap memory leak.", "verdict":
"CWE-401"}
```

Table 3: Dataset Partition and SFT Component Counts

Dataset Partition	Source Samples	LLVM IR Samples
Custom Synthetic Train	7,889	7,889
Custom Synthetic Validation	2,157	2,157
Custom Synthetic Test (Pool)	4,120	4,120
Synthset Eval (Headline Draw)	957	957
Deployable-733 (IR fits)	733	733
Realset Eval (BigVul Disjoint)	5,990	N/A
PrimeVul CVE Train (REALSFT)	3,128	N/A
Juliet SARD Training Aug.	7,980	7,980
Assembled SFT Mixture		
Custom Teacher Rationales	638	637
Juliet Augmentation	230	236
CWE Knowledge Cards	267	267
LLVM Knowledge Cards	N/A	230
Total Assembled SFT	**1,135**	**1,370**

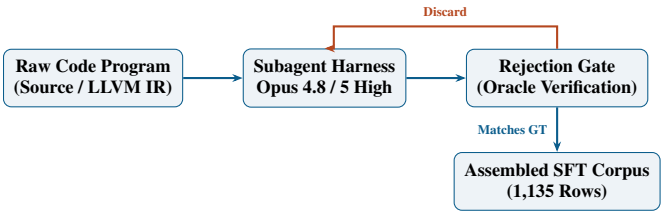


Figure 1: Rationale SFT Data Pipeline

Table 2 gives the exact system prompt and an example of the teacher's output.

2.5 Dataset Construction and Counts

Table 3 lists the sample counts for each split and for each part of the assembled fine-tuning mixture.

3 Models and Untuned Baselines

We evaluate five open models that differ in size, architecture and reasoning ability, with gpt-5.6-luna as the frontier reference. Table 4 lists their properties.

3.1 Metrics Used

Models are scored on detecting whether code is vulnerable, naming the right weakness class, and telling a real vulnerable function from its fixed version. Table 5 defines the ten metrics

Table 4: Model Registry and Properties. n/d means the provider has not disclosed the value.

Model	Arch.	Params	Active	Context
LFM2.5-8B-A1B	MoE	8.0B	1.5B	128k
LFM2.5-2.6B	Hybrid	2.6B	2.6B	128k
Qwen-3.5-9B	Dense	9.0B	9.0B	262k
Gemma-4-12B-it	Dense	12.0B	12.0B	262k
Coder-7B	Dense	7.0B	7.0B	128k
gpt-5.6-luna	Frontier	n/d	n/d	1M

used, with their column abbreviations and formulas.

Table 5: Complete Metrics Registry and Mathematical Definitions

Header	Full Metric Name	Mathematical Formula / Definition
A_{bal}	Balanced Accuracy	$\frac{1}{2}(R+S)$ where $R=\frac{TP}{TP+FN}$, $S=\frac{TN}{TN+FP}$
catch	Recall (TPR)	$R=\frac{TP}{TP+FN}$
FPR	False Positive Rate	$\text{FPR}=\frac{FP}{FP+TN}=1-S$
name	CWE Naming (Graded)	$d(p,g)$ over flagged true bugs, Eq. (2)
+fam	Family Naming	$d_{\text{fam}}(p,g)=\max_{q\in\{p\}\cup\text{anc}_{\leq 1}(p)} d(q,g)$
pair	Pair-Consistency (PCR)	$\frac{1}{\lvert P\rvert}\sum_{(v,s)\in P}\mathbf{1}[f(v)=1\wedge f(s)=0]$
split	Pair Split Rate	$\frac{1}{\lvert P\rvert}\sum_{(v,s)\in P}\mathbf{1}[f(v)\neq f(s)]$
cflip	Correct-Flip Rate	pair/split
ϕ	Within-Pair Correlation	Eq. (3)
tok	Mean Output Length	Average generated output tokens per program

Detection is scored with balanced accuracy (A_{bal}), the average of the true positive rate (R) and the true negative rate (S). A model that guesses at random, or flags everything, scores 0.50 no matter how many vulnerable samples the test set contains.

$$A_{\text{bal}}=\frac{1}{2}\left(\frac{TP}{TP+FN}+\frac{TN}{TN+FP}\right) \tag{2}$$

Each Realset pair has a vulnerable function v and its patched twin s. Pair-consistency (pair) is the share of pairs where the model flags v and clears s. It breaks into two parts. The split rate (split) is how often the model gives the two twins different verdicts, and the correct-flip rate (cflip) is how often those different verdicts point the right way, so pair = split×cflip. The within-pair correlation ϕ measures how strongly the verdicts on the two twins move together.

$$\phi=\frac{|P|n_{11}-n_1m_1}{\sqrt{n_1(|P|-n_1)m_1(|P|-m_1)}} \tag{3}$$

where $n_1=\sum f(v)$, $m_1=\sum f(s)$, and $n_{11}=\sum f(v)f(s)$.

Naming gives partial credit when the predicted weakness p sits close to the true weakness g in the MITRE CWE hierarchy [4].

$$d(p,g)=\begin{cases}1.0 & p=g\\ 0.5 & p\in\text{anc}(g)\vee g\in\text{anc}(p)\\ 0.25 & \text{parent}(p)=\text{parent}(g)\\ 0.0 & \text{otherwise}\end{cases} \tag{4}$$

3.2 Untuned Baseline Scores

Table 6 shows zero-shot scores for the untuned models on Synthset (957 programs), for both source and LLVM IR. In all results tables, **bold** marks the best small-model value in each column, and gpt-5.6-luna is listed as a reference point rather than a competitor.

Table 6: Untuned Baseline Benchmark Scores (Synthset 957). Naming is exact CWE. Coder-7B was run untuned on source only. Forced into a one-tag verdict format, this reasoning-oriented code model rarely flags anything and scores at chance.

Model	Rep	binAcc	catch	FPR	name	ms/ex
LFM2.5-8B-A1B	Source	0.514	0.354	0.325	0.037	719
LFM2.5-8B-A1B	LLVM	0.510	0.183	0.164	0.011	1074
LFM2.5-2.6B	Source	0.502	0.542	0.539	0.035	548
LFM2.5-2.6B	LLVM	0.510	0.273	0.254	0.076	960
Qwen-3.5-9B	Source	**0.643**	0.542	0.256	**0.504**	6762
Qwen-3.5-9B	LLVM	0.525	0.481	0.432	0.160	14476
Gemma-4-12B-it	Source	0.528	0.794	0.740	0.116	94
Gemma-4-12B-it	LLVM	0.511	0.773	0.753	0.089	1058
Coder-7B	Source	0.500	0.127	0.128	0.328	N/A
gpt-5.6-luna	Source	0.629	0.915	0.656	0.683	N/A
gpt-5.6-luna	LLVM	0.717	0.781	0.348	0.694	N/A

Table 7 shows the zero-shot prompt used for these baseline runs.

Table 7: Zero-Shot Inference Prompt Format

Zero-Shot Inference Prompt

```
You are a vulnerability classification system.  Analyze the {rep_label}
and respond with exactly one XML verdict tag.  If the code is vulnerable,
output <verdict>CWE-XXX</verdict>.  If the code is safe, output
<verdict>SECURE</verdict>.  Do not output anything else outside the tag.
Code ({rep_label}) {code}
```

4 Supervised Fine-Tuning (SFT)

Fine-tuning teaches the models a consistent output format and how to name weaknesses. It removes almost every unparseable answer, from up to roughly 6% (58 of 957) before tuning to under 0.1% after. The rank sweeps show that adapter size mainly drives naming skill, while the mix of training data mainly sets the false-positive rate.

4.1 Hardware and Cost

Fine-tuning ran on AWS, on four self-terminating g6e.xlarge instances (one NVIDIA L40S 48GB each) plus one on-demand p4d.24xlarge node (8x NVIDIA A100 40GB, 96 vCPUs, us-east-1). Startup scripts ran each job end to end, pulling code from S3, setting up the environment, training the LoRA adapter, evaluating with vLLM, uploading results and shutting the machine down. All sweeps and final runs together cost **$404 USD**.

4.2 SFT for Source C/C++

We test seven source-code configurations with LoRA ranks of 4, 8, 16 and 32, trained for 1 to 4 epochs on different data mixes.

4.2.1 Configurations

The configurations are listed below.

- **TERSE (Qwen-3.5-9B)** LoRA r16 for 3 epochs on 6,128 real and synthetic examples that contain only the verdict, with no reasoning.
- **Coder-7B terse (Qwen2.5-Coder-7B)** LoRA r16 for 3 epochs on the same 6,128 verdict-only examples. It has

the best Realset pair-consistency of any single model.

- **MIXED (Qwen-3.5-9B)** LoRA r16 for 3 epochs on 1,161 synthetic teacher rationales plus 3,356 verdict-only real examples, with a checkpoint saved after each epoch.
- **REALSFT (LFM2.5-2.6B)** LoRA r16 for 3 epochs on 3,128 clean real PrimeVul CVE examples.
- **Baseline SFT (LFM2.5-2.6B)** LoRA ranks 4, 8, 16 and 32 for 3 epochs on the assembled 1,135-sample synthetic mixture.
- **Expanded SFT (LFM2.5-2.6B)** LoRA r16 and r32 on a 1,641-sample corpus expanded with hard secure examples chosen to reduce false positives (45% secure).
- **sec55 SFT (LFM2.5-2.6B)** LoRA r16 on a corpus tilted toward precision (55% secure).

4.2.2 Results

Table 8 gives the full results on Synthset (957 synthetic programs) and Realset (2,995 real CVE pairs).

Table 8: Source SFT Results Across Benchmarks. name/+fam is graded CWE-tree naming with a one-hop family fallback, scored over flagged true vulnerabilities.

	Synthset (957)			Realset (2,995)			
Configuration	**binAcc**	**FPR**	**name/+fam**	**binAcc**	**FPR**	**pair**	**name/+fam**
Qwen MIXED ep3 (9B)	**0.626**	**0.224**	0.821/0.847	**0.514**	0.499	0.209	0.142/0.205
Qwen MIXED ep2 (9B)	0.608	0.260	0.775/0.807	0.514	0.493	0.202	0.132/0.189
Qwen TERSE ep3 (9B)	0.526	0.294	0.608/0.657	0.508	**0.211**	0.141	0.193/0.292
Coder-7B terse (7B)	0.512	0.365	**0.972/0.976**	0.510	0.510	**0.226**	0.209/0.312
REALSFT (2.6B)	0.521	0.396	0.104/0.152	0.504	0.426	0.178	**0.288/0.439**
SFT r16 baseline (2.6B)	0.543	0.686	0.593/0.628	0.503	0.668	0.156	0.120/0.154
SFT r32 baseline (2.6B)	0.515	0.602	0.678/0.703	0.504	0.642	0.180	0.108/0.139
r16+exp (45% sec)	0.543	0.474	0.636/0.665	0.508	0.636	0.167	0.113/0.142
r32+exp (45% sec)	0.542	0.503	0.697/0.723	0.507	0.663	0.165	0.110/0.140
sec55 (55% sec)	0.532	0.373	0.630/0.674	0.504	0.515	0.183	0.114/0.144
gpt-5.6-luna (Frontier)	0.629	0.656	0.765/0.769	0.545	0.502	0.156	0.278/0.362

4.2.3 What the Source Results Show

- **Teacher rationales drive real-code pair discrimination.** On the same Qwen-3.5-9B base, training on verdicts alone reaches a Realset pair-consistency of 0.141. Adding the teacher rationales raises it to 0.209, above the frontier's 0.156.
- **Real training code gives the best real-code naming.** REALSFT, the 2.6B model fine-tuned on real PrimeVul code, scores 0.288 on Realset naming, slightly above gpt-5.6-luna (0.278). It also has the lowest false-positive rate of the 2.6B models (0.426).
- **Recall-heavy synthetic mixes buy catch with false positives.** Models trained on mixes that favor recall catch 0.65 to 0.68 of real bugs, but they also flag 0.64 to 0.68 of safe functions, so their pair-consistency stays at 0.14 to 0.16.
- **The 2.6B model runs out of naming capacity.** Raising the LoRA rank from 4 to 32 improves its synthetic naming from 0.332 to 0.678, where it levels off. The 9B model reaches 0.821, a level the 2.6B does not approach.

4.3 SFT for LLVM IR

LLVM IR is the compiler's intermediate language. It drops comments and formatting and renames every variable into numbered registers, which in principle removes the surface variation that confuses small models.

4.3.1 The Cost of Long IR Inputs

IR is much longer than the source it comes from. Prompts grow by 6x to 9x, and backpropagation on an L40S uses about 1.9 MB of memory per token. To avoid running out of memory we cap training sequences at 16,384 tokens (batch size 1, gradient accumulation 32). This drops 61 of the 1,370 training samples (4.5%). At test time, 224 of the 957 programs (23.4%) are also longer than the cap, so IR models are scored on the Deployable-733 subset.

4.3.2 Configurations

We sweep LoRA ranks 4, 8, 16, 32, 64, 128 and 256 on LFM2.5-2.6B, training for 1 to 4 epochs.

4.3.3 Results

Table 9 gives the LLVM IR results on the Deployable-733 subset.

Table 9: LLVM IR SFT Rank Sweep Results (Deployable-733 Subset). Naming is exact CWE.

Model Rank	**Trainable**	**binAcc**	**catch**	**FPR**	**name**
Untuned Floor	0.0M	0.513	0.325	0.299	0.083
SFT r=4	6.1M	0.499	0.566	0.569	0.154
SFT r=8	12.2M	0.509	0.642	0.626	0.232
SFT r=16	24.5M	0.502	0.631	0.629	0.284
SFT r=32	48.9M	**0.516**	0.729	0.701	0.365
SFT r=64	97.8M	0.501	0.694	0.695	0.496
SFT r=128	195.6M	0.475	0.599	0.651	**0.648**
SFT r=256	391.3M	0.509	0.596	0.580	0.636
gpt-5.6-luna	Frontier	0.707	0.778	0.365	0.658

4.3.4 What the LLVM Results Show

- **Adapter rank drives IR naming up to rank 128.** Naming improves with every doubling of the LoRA rank, from 0.154 at rank 4 to 0.648 at rank 128, close to the frontier's 0.658.
- **Gains stop at rank 128.** Doubling again to rank 256 (391.3M trainable parameters, 14.4% of the base model) gives no further improvement (0.636 against 0.648), so rank 128 is the practical ceiling for the 2.6B model on IR.
- **IR changes the false-positive balance without improving small-model detection.** The frontier's balanced accuracy on IR reaches 0.707, 0.078 above its source score, yet the small models stay at chance (0.475 to 0.516) at every rank.

4.4 Source versus LLVM IR

Table 10 compares the two representations side by side.

Table 10: Source vs LLVM IR Representation Wrap-Up. Peak Naming Skill is exact CWE.

Metric / Dimension	**Source C/C++**	**LLVM IR**
Optimal LoRA Rank (2.6B)	r=16	r=128
Peak Naming Skill (2.6B)	0.684 (r32)	0.648 (r128)
Frontier Detection Accuracy	0.629	0.717
On-Device Detection Accuracy	0.543	0.509
Mean Input Prompt Length	988 tokens	22,777 tokens
Throughput Latency (ms/ex)	548 ms	960 ms
Real-World Test Availability	Yes (Realset)	No (Uncompilable)

Table 11: Source models versus the best LLVM IR model on the same Deployable-733 subset. **name** is exact CWE over flagged true vulnerabilities. On identical programs the small model detects and names better from source than from IR. Only the frontier gains from IR, and it does so by trading recall for fewer false positives.

Model (Deployable-733)	binAcc	catch	FPR	name
Qwen-3.5-9B untuned	0.505	0.046	0.036	0.588
Qwen-3.5-9B TERSE ep3	0.537	0.372	0.299	0.533
Qwen-3.5-9B MIXED ep3	**0.638**	0.497	0.222	0.781
Qwen2.5-Coder-7B terse	0.527	0.428	0.374	**0.975**
LFM2.5-2.6B SFT r4	0.528	0.756	0.701	0.315
LFM2.5-2.6B SFT r8	0.502	0.748	0.745	0.522
LFM2.5-2.6B SFT r16	0.554	0.786	0.679	0.614
LFM2.5-2.6B SFT r32	0.519	0.642	0.604	0.684
LFM2.5-2.6B r16+exp	0.551	0.593	0.492	0.644
LFM2.5-2.6B r32+exp	0.540	0.621	0.541	0.699
LFM2.5-2.6B sec55	0.550	0.474	0.374	0.634
LFM2.5-2.6B REALSFT	0.509	0.436	0.418	0.056
LFM2.5-2.6B LLVM r128 (best IR)	0.475	0.599	0.648	0.647
gpt-5.6-luna Source	0.623	0.900	0.654	0.663
gpt-5.6-luna LLVM IR	0.706	0.775	0.363	0.661

4.4.1 Source Models versus the Best IR Model on Deployable-733

Table 10 gives the overall picture, and Table 11 turns it into a controlled comparison. Every tuned source model and the best IR model (rank 128) are scored on the same Deployable-733 programs (Section 2.3), so neither side benefits from which programs were kept. All numbers are recomputed from the saved per-sample outputs.

- **Source beats IR for the small models.** The best source model (Qwen MIXED ep3, balanced accuracy 0.638) is far ahead of the best IR model (LFM LLVM r128, 0.475), and every 2.6B source configuration also beats it. Source naming reaches 0.684 for LFM r32 and 0.781 for Qwen MIXED, both above the 0.647 that IR reaches only at rank 128, an adapter four times the size of r32. On the same programs, compiling to IR gives the small model no advantage.
- **IR helps the frontier only by making it more cautious.** gpt-5.6-luna's balanced accuracy rises from 0.623 on source to 0.706 on IR. The gain comes from its false-positive rate falling from 0.654 to 0.363, while its recall drops from 0.900 to 0.775. The frontier model turns that trade into a net improvement, and the 2.6B model cannot.
- **Using IR well seems to need a larger model.** The frontier can read past surface tokens in IR, while the small model pays the length and rank costs of IR (Section 4.3) without any detection payoff. Coder-7B's 0.975 naming score is a side effect of its very short answers (13-token verdicts) on a detector that performs at chance, and should not be read as strong naming ability.

4.4.2 Why Synthset and Realset Diverge

Many models look strong on Synthset and ordinary on Realset. Qwen MIXED ep3 scores 0.626 balanced accuracy and 0.821 naming on synthetic code, then 0.514 and 0.142 on real code. The frontier drops too, from 0.629 to 0.545. Three differences between the benchmarks explain most of this gap.

- **Distribution match.** The synthetic training rationales and all of Synthset come from the same generator and its 104 defect patterns, so a model trained on those rationales sees Synthset programs that closely resemble its training data. Realset is real CVE code from BigVul that shares nothing with the training data (Section 2.3). The naming scores make this concrete. REALSFT, the only model trained on real PrimeVul code, has the worst Synthset naming (0.104) and the best Realset naming (0.288, above the frontier's 0.278). Each model is good at the kind of code it was trained on, which is a narrower skill than understanding vulnerabilities in general.
- **Whether one function is enough to decide.** Each synthetic program is self-contained, and the oracle confirmed that its bug fires inside that code, so the answer can be worked out from the function alone. Real CVE functions are cut out of larger projects and arrive without the macro definitions and callers that determine whether the vulnerable path can actually be reached. Often the correct label cannot be decided from the function by itself, which holds every small model near chance and the frontier to 0.545.
- **How fine the comparison is.** In Synthset, a vulnerable program is compared with an unrelated secure one, which is an easy contrast. In Realset, a vulnerable function is compared with its own fix, and the two differ only in the lines the patch changed. Telling a function from its nearly identical repair is much harder, which is why pair-consistency on real code stays low even for models that do well on Synthset.

The two benchmarks therefore measure different things. Synthset checks whether a model learned the defect patterns behind its training data. Realset checks whether that skill carries over to unseen production code that has lost its surrounding context and differs from its fix by a few lines. The drop between them is the synthetic-to-real transfer gap, and it is why this paper reports both and recommends a cascade with a human reviewer instead of trusting a synthetic score.

5 Direct Preference Optimization (DPO)

DPO was meant to widen the gap between real catches and false alarms. It trains on pairs of model answers in which the correct answer was confirmed by running the program.

5.1 Setup

Each training example contains a program x, a preferred answer y_w and a rejected answer y_l, where correctness is decided by running the program under sanitizers. The loss is the standard DPO objective.

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\Big[\log\sigma\Big(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\Big)\Big] \tag{5}$$

Here π_θ is the LoRA policy being trained, π_{ref} is the frozen SFT model it starts from, σ is the logistic sigmoid, and $\beta = 0.1$ controls how far the policy may move away from the reference.

5.2 Results

Table 12 shows the DPO results. The two Realset rows show where each SFT starting model sits on real code. The before-

and-after comparison is on PrimeVul (2,481 pairs) and Synthset (957 programs), the two sets where both the starting checkpoint and the DPO checkpoint were scored.

Table 12: Direct Preference Optimization Results Across Benchmarks

Model / Stage	Benchmark	binAcc	FPR	pair
REALSFT Start	Realset	0.504	0.426	0.178
sec55 SFT Start	Realset	0.504	0.515	0.183
sec55 SFT Start	PrimeVul	0.507	0.498	0.191
DPO Post-sec55 ep2	PrimeVul	0.507	0.448	0.187
sec55 SFT Start	Synthset	0.532	0.373	N/A
DPO Post-sec55 ep2	Synthset	0.565	0.277	N/A
DPO Post-sec55 ep3	Synthset	0.537	0.275	N/A

5.3 What DPO Changed

- **DPO moved the decision threshold without improving discrimination.** Training did what the loss asks and preferred the correct answer on 74% of candidate pairs. On PrimeVul, however, catch and false-positive rate fell together, from 0.51 to 0.46 and from 0.50 to 0.45. The model became more cautious overall and no better at telling vulnerable code from safe code.
- **Real-code pair-consistency stays at the wall.** Both Realset starting models sit at chance balanced accuracy (0.504). On PrimeVul, DPO moves pair-consistency only from 0.191 to 0.187, with balanced accuracy flat near 0.507.

6 Final Results and Best Models

Table 13 compares the best small-model configurations with gpt-5.6-luna on every metric.

Table 13: Final Benchmark Comparison of Top On-Device Models vs Frontier. name/+fam is graded CWE-tree naming with a one-hop family fallback. The LLVM r128 row is scored on the deployable-733 subset.

		Synthset (957)			Realset (2,995)			
Model Configuration	Role	Acc	FPR	Name/+fam	Acc	FPR	Pair	Name/+fam
Qwen MIXED ep3 (9B)	Generalist	**0.626**	**0.224**	0.821/0.847	**0.514**	0.499	0.209	0.142/0.205
Coder-7B terse (7B)	Real Leader	0.512	0.365	**0.972/0.976**	0.510	0.510	**0.226**	0.209/0.312
REALSFT (2.6B)	Real Namer	0.521	0.396	0.104/0.152	0.504	**0.426**	0.178	**0.288/0.439**
LLVM SFT r128 (2.6B)	IR Specialist	0.475	0.651	0.666/0.683	N/A	N/A	N/A	N/A
gpt-5.6-luna (Frontier)	Ceiling	0.629	0.656	0.765/0.769	0.545	0.502	0.156	0.278/0.362

6.1 Combining Models

Breaking pair-consistency into its two parts (pair $=$ split$\times$cflip) explains how the small models beat the frontier on pairs (0.226 against 0.156) despite lower balanced accuracy (0.510 against 0.545). The small models often give the two twins different verdicts (split $= 0.431$), but when they do, they pick the right direction only about half the time (cflip $= 0.524$). gpt-5.6-luna rarely separates the twins (split $= 0.222$, $\phi = 0.564$), but when it does it is usually right (cflip $= 0.701$).

Combining training data in one model also has a cost. Mixing synthetic and real supervision hurts real-code naming (0.142 for Qwen MIXED against 0.288 for REALSFT).

To get the strengths of different models in one system, we test voting ensembles and a two-stage detect-then-name cascade on the 2,995 Realset pairs. Table 14 describes each strategy and Table 15 reports the results.

Table 14: Combination Strategies Evaluated

Strategy	What it entails
Best Single Model	The strongest individual model, Coder-7B terse, run alone with no combination. The baseline any ensemble must beat.
Majority Vote (3)	Three specialists, Qwen MIXED ep3 (9B), Coder-7B terse (7B), and REALSFT (2.6B), each emit a vulnerable-or-secure verdict per function, and the function is flagged when at least two of the three agree.
Union	Flag a function vulnerable if any one of the three specialists (Qwen MIXED ep3, Coder-7B terse, REALSFT) flags it. Maximizes recall and floods false positives.
Intersection	Flag a function vulnerable only if all three specialists flag it. Maximizes precision and collapses recall.
Cascade (Detect + Name)	Two stages. A detector (Qwen MIXED ep3 for the 9B variant, or Coder-7B terse for the 7B) sets the flag, and each flagged finding is then routed to REALSFT (the 2.6B real-code model), which assigns the CWE class with a family-level fallback.
Frontier Ceiling	gpt-5.6-luna run alone, the reference line for a single hosted model.

Table 15: Real-World Ensemble and Detect-Then-Name Cascade Evaluation

Combination Strategy	Pair-Consistency	FPR	REAL Name/+fam
Best Single Model (Coder-7B)	0.226	0.510	0.209/0.312
Majority Vote (3 Models)	0.218	0.463	N/A
Union / Intersection	0.108 / 0.101	0.859 / 0.112	N/A
Cascade (Coder-7B detect + REALSFT name)	**0.226**	0.510	**0.290/0.442**
Cascade (Qwen MIXED ep3 detect + REALSFT name)	0.209	0.499	0.270/0.414
gpt-5.6-luna (Frontier Ceiling)	0.156	0.502	0.278/0.362

Majority voting (0.218) does not beat the best single detector (0.226). The cascade with Coder-7B as the detector matches that best pair-consistency and gives the best real-code naming of any setup (0.290/0.442), ahead of gpt-5.6-luna (0.278/0.362). Its false-positive rate stays at the detector's 0.510. The cascade works in two steps.

1. **Screening (Qwen-3.5-9B or Coder-7B).** A general detector decides which functions to flag. These models match the frontier's synthetic accuracy (0.626 against 0.629) and beat it on real-code pair-consistency (0.226 against 0.156).
2. **Naming (REALSFT 2.6B with family labels).** Each flagged function goes to the real-code specialist, which names the weakness and also reports its parent family. This raises real-code family naming to 0.442, against 0.362 for gpt-5.6-luna.

6.2 Limitations

- **Synthetic-to-real transfer gap.** Synthetic training does carry over to telling real functions from their fixes (pair-consistency 0.226 against 0.156 for the frontier), yet plain real-code detection stays at chance (0.50 to 0.51) for every small model.
- **Single functions lack context.** Real functions taken out of their projects often miss the macro definitions and callers that decide whether a bug can be reached, which limits how

well any single-function detector can do.

- **Mixture-of-experts models.** Standard LoRA cannot target the grouped expert weights in mixture-of-experts models such as LFM2.5-8B-A1B, so these models need full fine-tuning.
- **Model size ceiling.** Each fine-tuning job ran on one 48 GB L40S, which limits the study to models of about 12B parameters or smaller. Current state-of-the-art small models such as Qwen-3.6-27B or the 26B and 31B Gemma-4 models were out of reach, since a 27B model in bf16 does not fit on one L40S even before IR's long inputs are added. Larger models may have the capacity to learn the long-range structure in IR much better. The finding that small models cannot make use of IR therefore holds only up to about 12B and remains open above that.

6.3 Future Directions

Three directions could get past the real-code wall. The first is giving the model more than one function, through repository-level slicing, macro expansion and caller-callee graphs, so that labels become decidable. The second is building runtime harnesses for real CVE patches, which would allow verifier-based reinforcement learning on real code instead of on unverified commits. The third is scaling to 27B to 32B models with multi-node training, to test whether more capacity unlocks reasoning over IR.

7 Conclusion

We end by answering the three research questions.

RQ1 How far do 1 to 12B on-device models trail gpt-5.6-luna (high) on CWE classification, measured by balanced accuracy, naming, and pair-consistency, and how does the gap scale with model size and architecture?
On clean synthetic source code, the 9B model closes the gap completely. It matches gpt-5.6-luna on balanced accuracy (0.626 against 0.629) with one third of the false positives (0.224 against 0.656) and better weakness naming (0.821 against 0.765). On real source code, every small model sits at chance balanced accuracy (0.50 to 0.51), and gpt-5.6-luna reaches only 0.545. The small models still lead on real-code pair-consistency (0.226 against 0.156) and family-level naming (0.442 against 0.362).

Model size alone does not predict the gap. Untuned, the 9B Qwen (0.643) outscores the larger 12B Gemma (0.528), which flags almost everything (false-positive rate 0.740), and the mixture-of-experts LFM2.5-8B-A1B, with only 1.5B active parameters, scores about the same as the 2.6B hybrid model (0.514 against 0.502). After fine-tuning, the 9B model trained with rationale SFT is the one that reaches parity. The 7B Coder model, trained on verdicts alone, stays at 0.512, below the best 2.6B configuration (0.543), so the training data shapes the result as much as the parameter count does. Size shows most clearly in naming capacity. As LoRA rank grows, the 2.6B model's synthetic naming levels off at 0.678, well short of the 9B model's 0.821. On real code, size stops mattering. Every tuned model from 2.6B to 9B sits at chance balanced accuracy, while the 7B Coder model leads on pair-consistency (0.226) and the 2.6B REALSFT names real weaknesses best (0.288). Architecture also limited the comparison. Standard LoRA cannot tune the mixture-of-experts model, so it appears only as an untuned baseline.

RQ2 Does compiler-derived LLVM IR help or hurt a small model, relative to source, for the same task?
LLVM IR does not help small models. Their balanced accuracy on IR stays at chance (0.509), and IR inputs are 6x to 9x longer, which forces a 16,384-token cap that excludes 23% of evaluation programs. On the same Deployable-733 programs, the small models detect and name better from source (Section 4.4). IR does help the frontier, though only by changing its balance between catches and false alarms. It cuts gpt-5.6-luna's false-positive rate from 0.654 to 0.363 and lowers its recall from 0.900 to 0.775, and balanced accuracy rises to 0.706 because the drop in false positives outweighs the missed bugs. Small models can learn to name weaknesses from IR (0.648), but that ability stops improving at rank 128.

RQ3 How much does supervised fine-tuning on teacher rationales (rationale SFT), and preference-based reinforcement learning on top of it, close the gap over the untuned baselines?
Rationale SFT is the most effective lever. On the 9B model it raises synthetic detection from 0.508 to 0.626 (both scored on the training evaluation harness, where the untuned model often runs past its token budget, unlike the zero-shot harness behind the 0.643 in Table 6) and real-code pair-consistency from 0.099 to 0.209, while training on verdicts alone reaches only 0.141. DPO on top of fine-tuning makes the model uniformly more cautious, moving it along the ROC diagonal, and leaves the real-code wall in place.

8 References

References

[1] J. Fan, Y. Li, S. Wang, and T. N. Nguyen. A C/C++ code vulnerability dataset with code changes and CVE summaries. In *Proc. MSR*, 2020. https://doi.org/10.1145/3379597.3387501

[2] Y. Ding et al. Vulnerability detection with code language models How far are we? In *Proc. ICSE*, 2024. https://arxiv.org/abs/2403.18624

[3] NIST. Juliet C/C++ test suite, Software Assurance Reference Dataset (SARD). https://samate.nist.gov/SARD/test-suites/112

[4] MITRE. Common Weakness Enumeration (CWE). https://cwe.mitre.org

[5] D. Gnieciak and T. Szandala. Large language models versus static code analysis tools A systematic benchmark for vulnerability detection. *IEEE Access*, 2026. https://arxiv.org/abs/2508.04448

[6] Y. Li et al. Revisiting pre-trained language models for vulnerability detection, 2025. In *Proc. ACM Asia CCS*, 2026. https://arxiv.org/abs/2507.16887

[7] M. Al Atiiq, Y. Zhou, and S. Gehrmann. Dissecting the black box Circuit-level analysis of LLM vulnerability detection, 2026. https://arxiv.org/abs/2605.29901

[8] F. Müller-Uri and T. Pykälistö. Benchmarking large language models for vulnerability detection Comparing local and cloud LLMs. Technical Report, Lund University, 2026. https://lup.lub.lu.se/student-papers/search/publication/9229367

[9] Z. Lin and A. Mohaisen. From large to mammoth A comparative evaluation of large language models in vulnerability detection. In *Proc. NDSS*, 2025. https://dx.doi.org/10.14722/ndss.2025.241491

[10] R. Henareh. Fine-tuning small open-weight LLMs for cybersecurity. Master's thesis, KTH Royal Institute of Technology, 2026. https://kth.diva-portal.org/smash/get/diva2:2060365/FULLTEXT01.pdf

[11] L. Lu et al. Evaluating and enhancing the vulnerability reasoning capabilities of large language models, 2026. https://arxiv.org/abs/2602.06687

[12] A. Rondanini et al. LoRA-based parameter-efficient LLMs for continuous learning in edge-based malware detection, 2026. https://arxiv.org/abs/2602.11655

[13] A. V. Kozachok and A. V. Kozachok. A review of small language models for code and domain-specific language artifact generation. *Preprints.org*, 2026. https://www.preprints.org/manuscript/202606.1095

[14] J. Hu, X. Jin, Y. Zeng, Y. Liu, Y. Li, D. Du, K. Xie, and H. Zhu. QLPro Automated code vulnerability discovery via LLM and static code analysis integration, 2025. https://arxiv.org/abs/2506.23644

[15] S. Jin. Large language models for vulnerability detection in static code analysis A survey, 2025. https://ieeexplore.ieee.org/document/11082007

[16] Y. Xiong and T. Zhang. Sifting the noise A comparative study of LLM agents in vulnerability false positive filtering, 2026. https://arxiv.org/abs/2601.22952

[17] B. Ahmed, M. Khan, and R. Islam. SECVULEVAL Benchmarking LLMs for real-world C/C++ vulnerability detection, 2026. https://arxiv.org/abs/2505.19828

[18] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network, 2015. https://arxiv.org/abs/1503.02531

[19] Y. Kim and A. M. Rush. Sequence-level knowledge distillation. In *Proc. EMNLP*, 2016. https://arxiv.org/abs/1606.07947